ABBA2 Midpoint

Explicit duplicated-space step with arithmetic projection

Explicit palindromic operations

All ABBA and BM4 presets inherit `PalindromicMethod`. For signed $d = ch$, the elementary operations are

$$A_d(u, v, t) = (u + df(t, v), v, t), \qquad B_d(u, v, t) = (u, v + df(t, u), t), \qquad T_d(u, v, t) = (u, v, t + d).$$

Only T advances time. A coefficient $c = 1/2$ means duration $h/2$; there is no additional half inside A or B. The optional exact C flow rotates the copy difference and requires an independent coupling object. Both BM4 presets omit C.

In chronological order, standard ABBA2 is $A_{h/2}, B_{h/2}, T_h, B_{h/2}, A_{h/2}$. The distinct symmetric variant $T_{h/2}, A_{h/2}, B_{h/2}, B_{h/2}, A_{h/2}, T_{h/2}$ evaluates every spatial shear at the midpoint time. Both exhibit second order on the tested nonautonomous rotation. Arbitrary palindromes do not inherit fourth or sixth order, and an arithmetic projection need not be reversible or symplectic.

`Composition` denotes an explicit tuple of operations. Consecutive time translations combine as $T_a T_b = T_{a+b}$. The generic `PalindromicImplicit` and `PalindromicMidpoint` methods select one exterior Hairer projection or a final arithmetic mean. The signed spatial flows and passive momentum normalization are retained; observers reconstruct their groups from elementary traces without replay. NumPy and JAX execute the same explicit operations and differentiate their retained shear inputs. See `docs/models/palindromic/simulation/palindromic-operations.md`.

1 Definition

For $\dot{z} = f(t, z)$, $z \in \mathbb{R}^m$, duplicate the accepted state on the diagonal, $u_0 = v_0 = z_n$, and write $s = h/2$. The endpoint-time ABBA stages are

$$u_1 = u_0 + sf(t_n, v_0), \qquad v_1 = v_0 + sf(t_n, u_1), \tag{1}$$

$$v_f = v_1 + sf(t_n + h, u_1), \qquad u_f = u_1 + sf(t_n + h, v_f). \tag{2}$$

The physical output is the midpoint of the copies,

$$z_{n+1} = \frac{u_f + v_f}{2}.$$

"Midpoint" therefore refers to this arithmetic projection; the method is not the implicit-midpoint Runge–Kutta rule.

2 State and energy configurations

One endpoint-time A-B-B-A map followed by a spatial arithmetic mean. Energy tracking is optional and passive. The four accepted shear stages update the normalized passive momentum; no nonlinear solve is used. The common state conventions are specified below.

3 Symmetry, order, and geometry

The four shears form an explicit self-adjoint map on the duplicated phase space when the signed step reverses together with its endpoint times. Starting from the diagonal and taking the arithmetic

mean yields a consistent symmetric method with designed global order two for sufficiently smooth dynamics.

The duplicated ABBA map is symplectic for its split Hamiltonian. Arithmetic projection, however, is not Hairer's symmetric projection and does not in general make the closed physical map symplectic. The copy-separation quantity

$$d_n = \|u_f - v_f\|_\infty$$

is therefore retained as a diagnostic. It is neither an error estimator nor a convergence residual.

4 Work and time dependence

One step performs four vector-field evaluations. The evaluations at t_n and $t_n + h$ are part of the non-autonomous definition. The common fixed-grid runner chooses a uniform main grid independently of requested output times; off-grid output uses shadow steps that do not alter the accepted main trajectory or its diagnostics.

5 Implementation correspondence

The public class is in `src/methods/palindromic/abba2_midpoint.py`. It uses the common `midpoint_step`, the uncoupled ABBA pair in `abba2_implicit.py`, and exposes retained quadrature data to the same energy observer as implicit ABBA and BM4.

The public names, effective numerical defaults and exports from `simulation` remain unchanged. Internal imports use the defining modules; old import adapters have been removed. The ordinary implicit path is `advance`, `solve_projection`, `compose`, then explicit $A/B/T/C$ operations. Arithmetic midpoint variants use one explicit average instead of a nonlinear projection. Adjoint stages evaluate at the start of their signed substep; direct stages evaluate at its end.

Newton and Broyden share the spatial equations and convergence services. Exact particle-block tangents, passive energy and optional observation use retained shear inputs; energy is evaluated only after accepting the spatial map, with the existing factor 1/2 for physical κ. Time and momentum never enter the spatial root. The new internal family name does not enable the retired full time/momentum projection.

Current module responsibilities and diagrams are documented in `docs/models/palindromic/simulation/palindromic-simulation-architecture.md`.

6 Physical state and endpoint energy observations

Accepted states contain one physical vector $Z \in \mathbb{R}^{dN}$ and scalar time. GC vectors have $2N$ entries and FC vectors have $4N$; doubled workspaces are transient. InitialConditions supplies values and layout during preparation only. The numerical base retains functions, metadata, run status and copy count; execution owns the initial snapshot and output schedule. Supplied Jacobians are independent particle blocks (N, d, d); finite differences use dense matrices.

The generic `EnergyObserver` evaluates only

$$H_n = H(t_n, Z_n), \quad H_{n+1} = H(t_{n+1}, Z_{n+1}), \quad (\partial_t H)_n, \quad (\partial_t H)_{n+1}, \quad \Delta H_n = H_{n+1} - H_n.$$

It also evaluates these physical functions at requested saved samples, without consuming internal stages or integrating a momentum coordinate. The derivative is positive, analytic and partial at fixed state. GC differentiates its normalized Hamiltonian potential with `dt=1`; spatial splines and

Fourier time factors retain their original scaling. This is not the secant $\Delta H_n/h$. Variation of a time-dependent Hamiltonian is not a numerical conservation error.

Specialized historical generalized-energy studies separately reconstruct $\dot{\kappa} = -\partial_t H$ from retained numerical stages. Their signed weights, physical normalization and archived diagnostics keep their original meaning. They are not part of generic energy observation. Host and JAX execution share the fixed numerical map and physical-only accepted state. JAX evaluates endpoint energy on the device without a momentum accumulator.

7 Optional JAX execution

An optional JAX driver executes the same numerical map on one selected CPU or GPU, with particle operations batched inside a compiled sequential time loop. It reuses the canonical coefficients and prepared potential splines. Physical-state tolerance scaling, signed stage times, passive energy weights and independent shortened output maps retain their definitions above. Applicable Newton systems use independent particle blocks; an explicitly selected Broyden iteration retains its global secant matrix. Nonlinear iteration limits and failure checks remain active on the device.

Both execution routes use double precision and return the same physical solution layout. Floating-point reassociation can produce round-off differences without changing the designed order or geometric properties. First-call compilation is separate from warmed integration timings, which include transfers and output construction. The sibling runtime document and shared execution contract specify supported callbacks and exact device scope.